

PROGRAMMA A SIATI FINII

sabato 12 febbraio 2022 17:34

Cos'è un linguaggio? Per definire che cos'è un linguaggio ci serve il concetto di **stringa**.
Per definire cos'è una stringa definiamo cos'è un alfabeto.

STRINGA SU $\Sigma \rightarrow$ sequenza finita di simboli in Σ

ALFABETO: insieme finito di simboli

Esempio:

$\Sigma = \{a, b, c, d, \dots, z\}$ STRINGA SU $\Sigma = "aabcada"$

si può indicare l'insieme di tutte le stringhe costruibili su un alfabeto: Σ^*
questo è un insieme infinito perché è sempre possibile costruire una nuova stringa a partire da una precedente, aggiungendo un nuovo simbolo.

infinito $\rightarrow \Sigma^* = \{ "", "a", "b", \dots, "z", "aa", "bb", \dots \}$

finito $\rightarrow A^*$

LA stringa nulla è dentro questo insieme: ""

• **lunghezza di una stringa**

• **stringa nulla** = stringa senza alcun simbolo. Esempio = "". ha lunghezza zero.

$\epsilon \rightarrow \epsilon \in \Sigma^*$

• **sottstringa** = una stringa è una sottstringa di un'altra se i caratteri che compongono la stringa più piccola compaiono nel giusto ordine consecutivamente nella stringa più grande.

• **concatenazione di stringa**.

• **ordine lessico grafico** = "alfa" < "beta" // alfa precede beta! "alfa" < "bit"

a precede b

• **stringa order** = una stringa più corta precede sempre una stringa più lunga, e se le stringhe hanno diverse lunghezze non vale più la regola precedente.

(vince solo l'ordine alfabetico se le stringhe hanno len uguale)

"alfa" < "beta"
a precede b

alfa
len=4

bit
len=3

> (Anche se "a" precede "b")

A questo punto siamo in grado di definire cos'è un **linguaggio sopra un certo alfabeto** Σ : è un sottoinsieme di stringhe sopra l'alfabeto.

$L \subseteq \Sigma^*$

Esempio:

$\Sigma = \{0, 1\}$

$\Sigma^* = \{ \epsilon, 0, 1, 00, 01, 10, 11, 000 \dots \}$

STRINGA vuota = ""

L è un qualunque sottoinsieme di queste stringhe!

$L = \{ w \in \Sigma^* \mid w \text{ codifica in binario un numero divisibile per } 3 \}$

$$L = \{ 0, 11, 110, 1001, \dots \} \quad \begin{matrix} \notin L \\ \notin L \\ \notin L \\ \notin L \end{matrix}$$

IL Linguaggio rappresenta le istanze che soddisfano un certo quesito.

↓
un certo numero è divisibile per 3?

[PROGRAMMA A STATI FINITI]

$m \in \mathbb{N} \rightarrow (1011010)_2 \rightarrow "1011010" \in L?$ se sì, allora m è divisibile per 3.

ORA, esiste una relazione tra i linguaggi e i problemi!

IN questo esempio abbiamo 2 bit di memorizzazione: $\{0, 1\}$ e possiamo codificare 4 possibili numeri: 00, 01, 10, 11. Perciò possiamo definire degli stati interni:



Nella macchina possiamo codificare delle istruzioni che portano da uno stato all'altro sulla base della lettura dei simboli di una certa stringa.

ESEMPIO di PROGRAMMA A STATI FINITI!

• IL PIÙ piccolo programma che riconosce se un numero codificato in binario è o no divisibile per 3!

0	1	2	3	4	5	6	7	8	9	10	11	12	
00	01	10	11	100	101	110	111	1000	1001	1010	1011	1100	

✓

✓

✓

✓

✓

se dovessi implementare un algoritmo di divisione, impiegherei molta memoria e non è il più piccolo programma.

come potrebbe essere fatto il più piccolo programma?

In questa sequenza i numeri divisibili per 3 sono disposti in modo regolare.

o sono numeri divisibili per 3, oppure sono a distanza 1 da un numero divisibile per 3, oppure a distanza 2.

quindi il nostro programma avrà 3 stati!!! (0, 1, 2, 3)

$k = \text{numero}$

2 bit $\rightarrow$ 4 stati
" "
 2^2

INPUT: 1011001100110011

q_0
↓
"stato fortunato"
numero divisibile
per 3.
||
 $3k$

q_1
↓
stato in cui
i numeri sono a
distanza 1 da
un numero divisibile
per 3
||
 $3k+1$

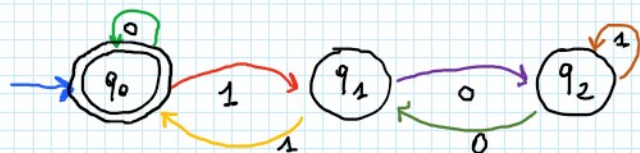
q_2
↓
stato in cui
i numeri sono a
distanza 2 da
un numero divisibile
per 3
||
 $3k+2$

qual'è lo stato iniziale?

Facciamo finta che la nostra sequenza di bit parta da uno zero, che non altera il valore del numero.

0 1011001100110011

la sequenza verrà letta da sinistra a destra!



- il primo numero che leggiamo è 0, che è un numero divisibile per 3, e quindi siamo nello stato q_0 : $(3k)$.

ora, dallo stato q_0 , leggiamo un'altra cifra, che può essere 0 o 1.

che succede se leggiamo il bit 0?

- se aggiungiamo uno zero in coda ad un numero binario, significa raddoppiare quel numero

$2(3k) + 0$. quindi se sono in questo stato q_0 e leggo uno zero, rimarrò in questo stato!

Facciamo finta che siamo in questo stato q_0 perché abbiamo letto 11 che è un numero divisibile per 3, se io ora leggo uno zero, diventa 110 diventa il

doppio, $2(3k) + 0$

$6k + 0$

$6k = 3k'$

che è ancora un numero divisibile per 3.

che succede se leggo il BIT 1?

Raddoppierò il valore precedente e ci raggiungerò un 1: $2(3K) + 1$

cioè $3K' + 1$. questo è un numero a distanza 1 dal multiplo di 3.

• quindi se leggo 1 devo andare in q_1 .

q_1 è lo stato di un numero che è a distanza 1 dal multiplo di 3:

ora se da qui leggo uno zero devo fare la stessa cosa:

$$2(3K + 1) + 0 = 6K + 2$$

↳ il numero che esce fuori è a distanza 2 da un multiplo di 3.

• quindi se leggo uno zero da q_1 , vado in q_2 .

• e se da q_1 leggo un 1?

$$2(3K + 1) + 1 = 6K + 3 = 3K' + 3$$

↳ che è un multiplo di 3.
quindi torno in q_0 .

q_2 è lo stato in cui un numero è a distanza 2 da un multiplo di 3.

ora, se da qui leggiamo uno zero?

$$2(3K + 2) + 0 = 6K + 4 = (6K + 3) + 1 = 3K' + 1$$

↓
multiplo di 3

quindi torno in q_1 → stato a distanza 1 da un multiplo di 3.

e se leggo un uno?

$$2(3K + 2) + 1 = 6K + 5 = (6K + 3) + 2$$

↳ numero a distanza 2 da un multiplo di 3.

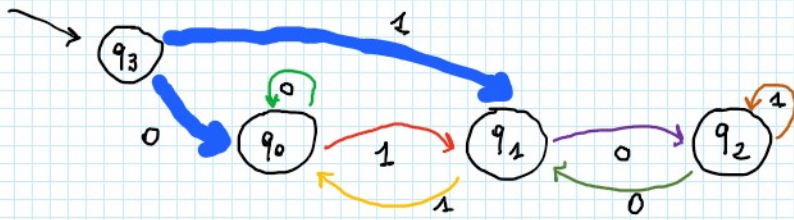
e quindi resto in q_2 .

ora, tutti gli stati hanno la loro freccia uscente per ciascuno dei bit che posso leggere.

L'idea è che, se l'ultimo simbolo letto dalla stringa mi porta nello stato q_0 , allora eccetto la stringa ed è un numero divisibile per 3.
Lo stato finale deve essere q_0 !

però la sequenza potrebbe essere nulla, e se è nulla noi ci fermiamo nello stato q_0 , ma non è un numero divisibile per 3, si **ovvia** aggiungendo uno stato!

$$L = \{ w \in \{0, 1\}^* \mid w \text{ codifica un numero divisibile per } 3 \}$$



* Esclude la stringa vuota.

$F = \{ q_0 \}$ q_3 stato iniziale